

Lab3Algorithms

September 1, 2026

1 AVL Tree Insertions and Rotation (Weiss Book Logic)

RIGHT-ROTATION(k1, k2)

```
1 TEMP = k1.rchild
2 k1.rchild = k2
3 k2.lchild = TEMP
4 k2.height = max(HEIGHT(k2.lchild), HEIGHT(k2.rchild)) + 1    k2 first
5 k1.height = max(HEIGHT(k1.lchild), k2.height) + 1
6 return k1
```

LEFT-ROTATION(k1, k2)

```
1 TEMP = k1.lchild
2 k1.lchild = k2
3 k2.rchild = TEMP
4 k2.height = max(HEIGHT(k2.lchild), HEIGHT(k2.rchild)) + 1
5 k1.height = max(k2.height, HEIGHT(k1.rchild)) + 1
6 return k1
```

DOUBLE-RIGHT-ROTATION(k1, k3)

```
1 k2 = k1.rchild
2 k1 = LEFT-ROTATION(k2, k1)
3 k3 = RIGHT-ROTATION(k1, k3)
4 return k3
```

DOUBLE-LEFT-ROTATION(k1, k3)

```
1 k2 = k1.lchild
2 k1 = RIGHT-ROTATION(k2, k1)
3 k3 = LEFT-ROTATION(k1, k3)
4 return k3
```

INSERT(x, T)

```
1 if T == NIL
2     T = NEW-NODE(x); T.height = 0; return T
3 if x < T.element
4     T.lchild = INSERT(x, T.lchild)
5     if HEIGHT(T.lchild) - HEIGHT(T.rchild) == 2
6         if x < T.lchild.element
7             T = RIGHT-ROTATION(T.lchild, T)    left-left
```

```

8         else T = DOUBLE-RIGHT-ROTATION(T.lchild, T)           left-right
9     elseif x > T.element
10         T.rchild = INSERT(x, T.rchild)
11         if HEIGHT(T.rchild) - HEIGHT(T.lchild) == 2
12             if x > T.rchild.element
13                 T = LEFT-ROTATION(T.rchild, T)                 right-right
14             else T = DOUBLE-LEFT-ROTATION(T.rchild, T)          right-left
15 T.height = max(HEIGHT(T.lchild), HEIGHT(T.rchild)) + 1
16 return T

```

2 AVL Tree Search, Traversal and Delete (Claude Sonnet 5 Low Logic)

FIND(x, T)

```

1  if T == NIL or x == T.element
2      return T
3  if x < T.element
4      return FIND(x, T.lchild)
5  return FIND(x, T.rchild)

```

FIND-MIN(T)

```

1  if T == NIL return NIL
2  while T.lchild != NIL
3      T = T.lchild
4  return T

```

INORDER(T)

```

1  if T != NIL
2      INORDER(T.lchild); VISIT(T.element); INORDER(T.rchild)

```

BST-DELETE(x, T)

```

1  if T == NIL return T
2  if x < T.element    T.lchild = BST-DELETE(x, T.lchild)
3  elseif x > T.element T.rchild = BST-DELETE(x, T.rchild)
4  else
5      if T.lchild != NIL and T.rchild != NIL
6          s = FIND-MIN(T.rchild)
7          T.element = s.element
8          T.rchild = BST-DELETE(s.element, T.rchild)
9      else T = (T.lchild != NIL) ? T.lchild : T.rchild
10 return T

```

AVL-DELETE(x, T)

```

1  T = BST-DELETE(x, T)
2  if T == NIL return T
3  T.height = max(HEIGHT(T.lchild), HEIGHT(T.rchild)) + 1
4  b = HEIGHT(T.lchild) - HEIGHT(T.rchild)

```

```
5  if b == 2
6      if HEIGHT(T.lchild.lchild)  HEIGHT(T.lchild.rchild)
7          T = RIGHT-ROTATION(T.lchild, T)
8      else T = DOUBLE-RIGHT-ROTATION(T.lchild, T)
9  elseif b == -2
10     if HEIGHT(T.rchild.rchild)  HEIGHT(T.rchild.lchild)
11         T = LEFT-ROTATION(T.rchild, T)
12     else T = DOUBLE-LEFT-ROTATION(T.rchild, T)
13 return T
```